EX NO: 1 **Study of Azure DevOps**

AIM:

To study how to create an agile project in Azure DevOps environment.

About Azure:

- It provides on-demand services for:
  - Computing
  - Storage
  - Networking
  - Databases
- Azure follows a pay-as-you-go pricing model.
- Users pay only for the resources they use.
- Azure services are hosted in global data centers for:
  - High availability
  - Scalability
  - Reliability
  - Security

Key Services :

- → Azure App Service – Hosts web and mobile applications without managing servers.
- → Azure SQL Database – Managed relational database service.
- → Azure Blob Storage – Stores unstructured data like images and files.
- → Azure Active Directory – Handles user authentication and role-based access.
- → Azure Monitor – Tracks performance, logs, and sends alerts.
- → Azure DevOps – Enables pipelines for automated build and deployment

Result:

Microsoft Azure cloud platform and its key services were studied successfully.

Ex No:2 CREATING A PROJECT IN AZURE

Aim:

To create and configure a cloud project in Microsoft Azure for the Canteen Online vehicle rental management system

Procedure:

Login

Login to Azure Portal using a Microsoft account.

2. Create Resource Group

Create a Resource Group named **'vehicle-rental-RG'** to hold all project resources.

3. Create App Service

Create an Azure App Service named **'vehicle-rental-app'** with Node.js runtime.

4. Create SQL Database

Create an Azure SQL Database named **'vehicleRentalDB'** with:

- Server credentials
- Database username and password
- Firewall configuration for secure access

5. Create Storage Account

Create an Azure Blob Storage account named **'vehiclerentalstorage'** and add a container named **'vehicle-images'** to store vehicle photos and documents.

6. Application Settings

Add application environment variables for:

- Database connection string
- API keys

- Storage account keys
- Application secrets

7. Configure GitHub Actions

Configure GitHub Actions for CI/CD to automatically build and deploy the application whenever code is pushed to the repository.

8. Verify Deployment

Verify deployment by accessing the live URL provided by Azure App Service and check whether:

- Vehicle listings are displayed
- Booking system works properly
- Database connectivity is successful
- Images load correctly from storage

Result:

Azure project for the online vehicle rental management system was successfully created with App Service, SQL, Blob Storage and CI/CD pipeline configured and deployed.

Ex No: 3 FUNCTIONAL AND NON-FUNCTIONAL REQUIREMENTS

Aim

To create the Functional and Non Functional Requirement for the Online vehicle rental management

Procedure:

Functional Requirements

1. User Registration

• The system shall allow users to register using email, phone number, or social login.

2. User Authentication

• The system shall allow users to log in and log out securely.

3. Profile Management

• The system shall allow users to create, view, and update their personal profiles.

4. Vehicle Browsing

• The system shall allow users to browse available vehicles with details such as model, type, price, and availability.

5. Vehicle Search and Filtering

• The system shall allow users to search and filter vehicles based on category, price range, location, and availability.

6. Vehicle Booking

• The system shall allow users to book vehicles by selecting rental date, time, and duration.

7. Booking Modification

• The system shall allow users to modify or cancel their bookings before confirmation.

8. Payment Processing

• The system shall allow users to make secure online payments through multiple payment methods.

Non-Functional Requirements

1. Performance

• The system shall provide a response time of less than 3 seconds under normal operating conditions.

2. Scalability

• The system shall be able to scale efficiently to support an increasing number of users, vehicles, and bookings.

3. Security

• The system shall ensure secure authentication, encrypted transactions, and role-based access control.

4. Reliability & Availability

• The system shall be available 24/7, excluding scheduled maintenance, and ensure data consistency during failures.

5. Usability

• The system shall provide a simple, intuitive, and user-friendly interface accessible on both desktop and mobile devices.

6. Maintainability

•

The system shall be easy to maintain and update with minimal downtime.

7. Compatibility

• The system shall support multiple browsers and operating systems.

Result:

Therefore the functional and non functional has been exected successfully

Ex No: 4 User story in Azure Devops

Aim:

To create User Stories in Azure Devops6*6*/6933-9/

Procedure:

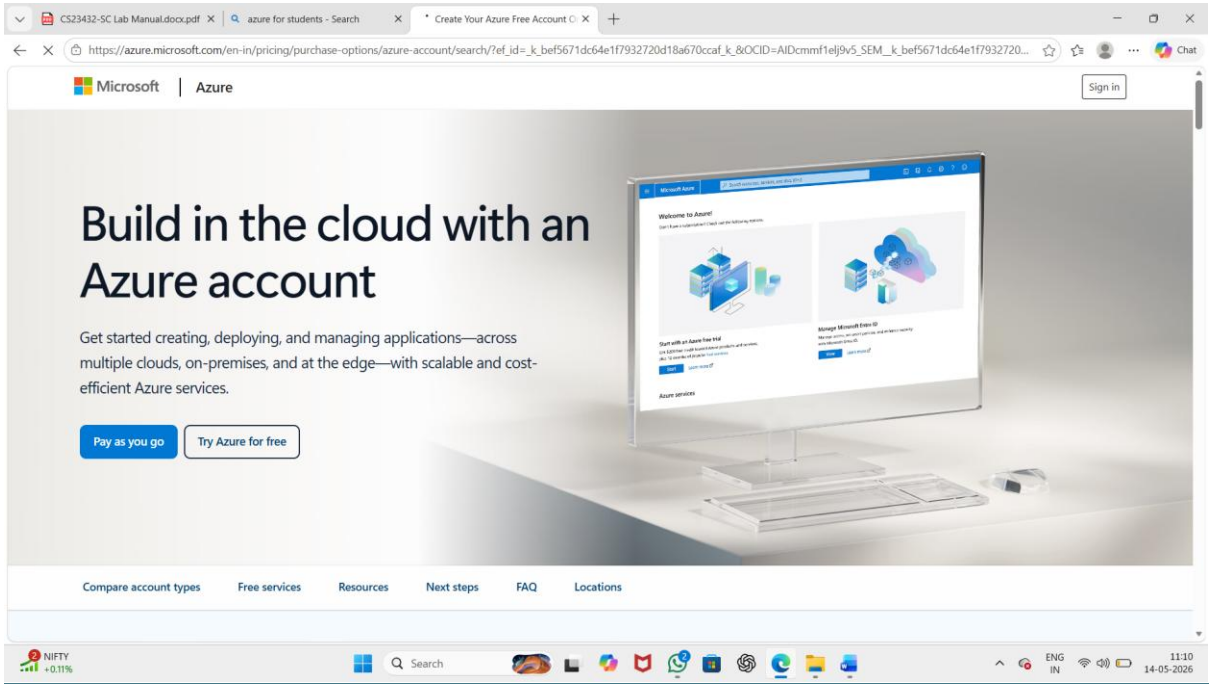
1. Open your web browser and go to the Azure website:

https://azure.microsoft.com/en-in Sign in using your Microsoft account

credentials. If you don't have an account, you'll need to create one.

2. If you don’t have a Microsoft account, you can sign up for

https://signup.live.com/?lic=1



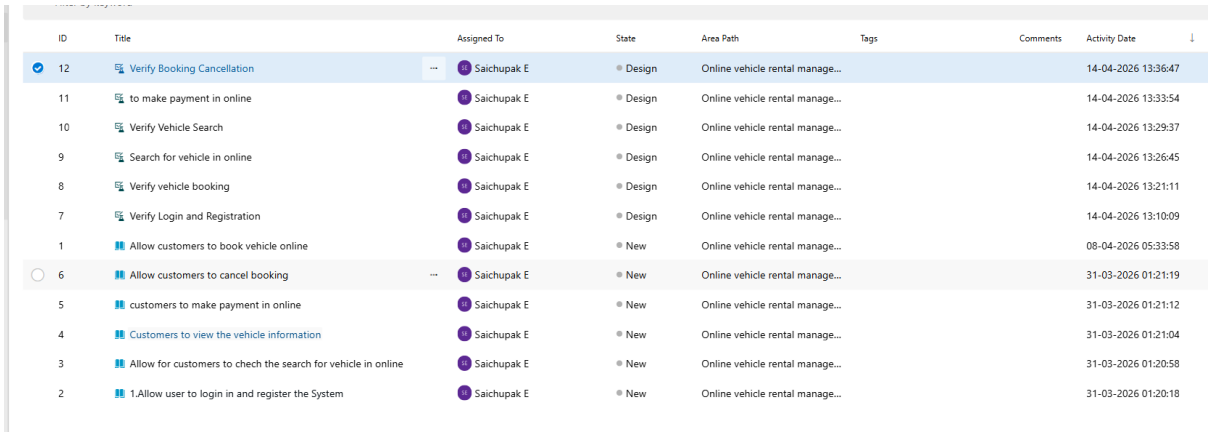
User Stories – Functional Requirements (Online Vehicle Rental Management System)

1. As a customer, register and login, I can access my account securely.
2. As an admin, add/edit vehicle details, so customers can choose available vehicles easily.
3. As a customer, view the vehicle list, I can choose a suitable vehicle for rent.
4. As a customer, book a vehicle online, I can reserve the vehicle before visiting.

5. As a customer, track my booking status, I know whether my vehicle is confirmed or available.
6. As a customer, view booking history, I can rent my favourite vehicle again easily.
7. As a customer, make online payment, the transaction is cashless and fast.
8. As a customer, receive notifications, I am informed about booking confirmations and updates.
9. As a rental staff, view pending bookings, I can prepare and manage vehicle delivery on time.

User stories -Non Functional requirement(Online vehicle rental management system)

1. The system should provide fast response and processing time for users.
2. The system should ensure secure access and protect user data.
3. The system should be available at all times with minimal downtime.
4. The system should support a large number of users without performance issues.
5. The system should provide an easy and user-friendly interface.



ID	Title	Assigned To	State	Area Path	Tags	Comments	Activity Date
12	Verify Booking Cancellation	Saichupak E	Design	Online vehicle rental manage...			14-04-2026 13:36:47
11	to make payment in online	Saichupak E	Design	Online vehicle rental manage...			14-04-2026 13:33:54
10	Verify Vehicle Search	Saichupak E	Design	Online vehicle rental manage...			14-04-2026 13:29:37
9	Search for vehicle in online	Saichupak E	Design	Online vehicle rental manage...			14-04-2026 13:26:45
8	Verify vehicle booking	Saichupak E	Design	Online vehicle rental manage...			14-04-2026 13:21:11
7	Verify Login and Registration	Saichupak E	Design	Online vehicle rental manage...			14-04-2026 13:10:09
1	Allow customers to book vehicle online	Saichupak E	New	Online vehicle rental manage...			08-04-2026 05:33:58
6	Allow customers to cancel booking	Saichupak E	New	Online vehicle rental manage...			31-03-2026 01:21:19
5	customers to make payment in online	Saichupak E	New	Online vehicle rental manage...			31-03-2026 01:21:12
4	Customers to view the vehicle information	Saichupak E	New	Online vehicle rental manage...			31-03-2026 01:21:04
3	Allow for customers to chech the search for vehicle in online	Saichupak E	New	Online vehicle rental manage...			31-03-2026 01:20:58
2	1.Allow user to login in and register the System	Saichupak E	New	Online vehicle rental manage...			31-03-2026 01:20:18

Result:

User stories for 9 functional and 5 non-functional requirements were written with all attributes including Story ID, Role, Goal, Benefit, Acceptance Criteria, and Priority.

Ex No: 5 Sequence Diagram

Aim:

To design a Sequence Diagram for the Online vehicle rental management system

Procedure:

::: mermaid

sequenceDiagram

actor Customer

participant System

participant Vehicle

participant Booking

participant Payment

participant Rental

participant Location

%% Search Vehicle

Customer ->> System: Login / Register

Customer ->> System: Search Vehicle

System ->> Vehicle: Check Availability

Vehicle -->> System: Available Vehicles List

System -->> Customer: Show Vehicles

%% Booking

Customer ->> System: Select Vehicle

Customer ->> System: Enter Dates & Location

System ->> Booking: Create Booking

Booking -->> System: Booking Confirmation

%% Payment

System ->> Payment: Request Payment

Customer ->> Payment: Make Payment

Payment -->> System: Payment Success

%% Rental Process

System ->> Rental: Start Rental

Rental -->> Customer: Vehicle Pickup Details

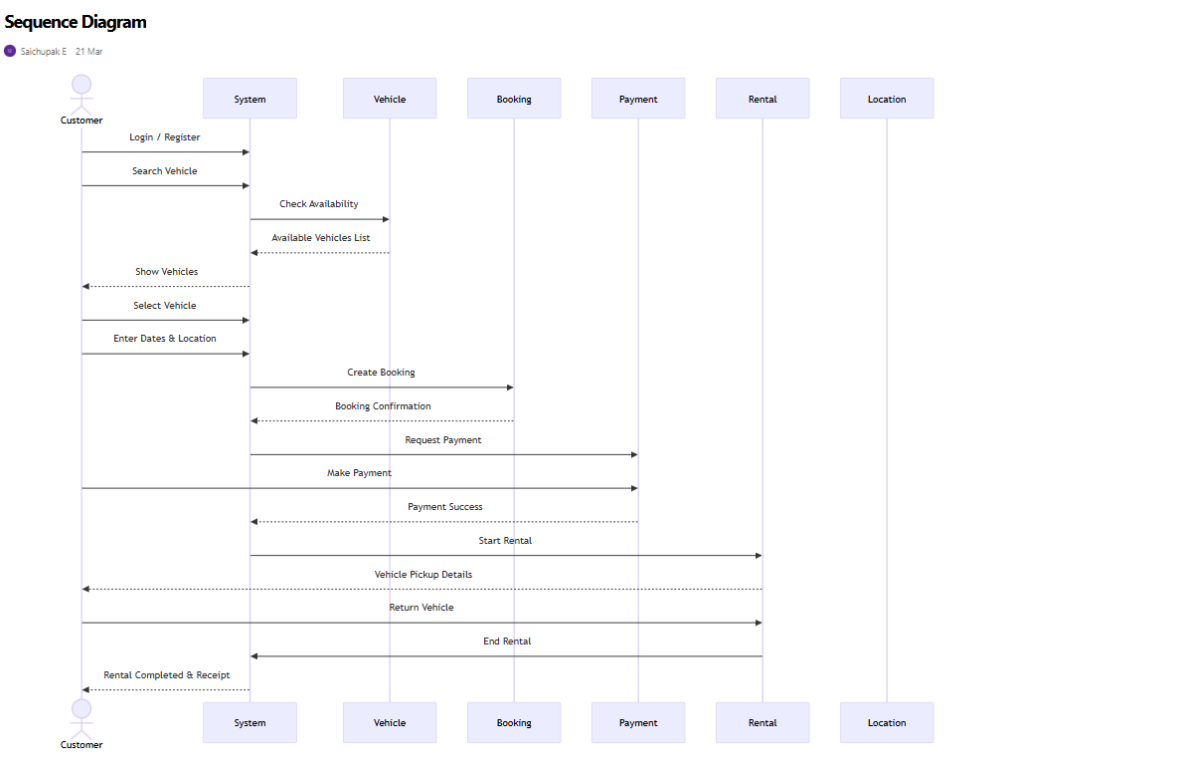
Customer ->> Rental: Return Vehicle

Rental ->> System: End Rental

%% Completion

System -->> Customer: Rental Completed & Receipt

:::



Result:

Therefore the Sequence diagram is executed successfully.

Ex No: 6 Class Diagram

Aim:

To create the Class Diagram for the Online vehicle rental management system

Procedure:

::: mermaid

classDiagram

class Admin {

+int adminId

+String name

+login()

+manageBus()

+manageRoute()

+manageBooking()

}

class Bus {

+int busId

+String busNumber

+int capacity

+String type

+addBus()

+updateBus()

}

class Driver {

+int driverId

+String name

+String licenseNumber

+assignBus()

}

class Route {

+int routeId

+String source

+String destination

+int distance

+addRoute()

}

class Schedule {

+int scheduleId

+String departureTime

+String arrivalTime

+createSchedule()

}

class Passenger {

+int passengerId

+String name

+String contact

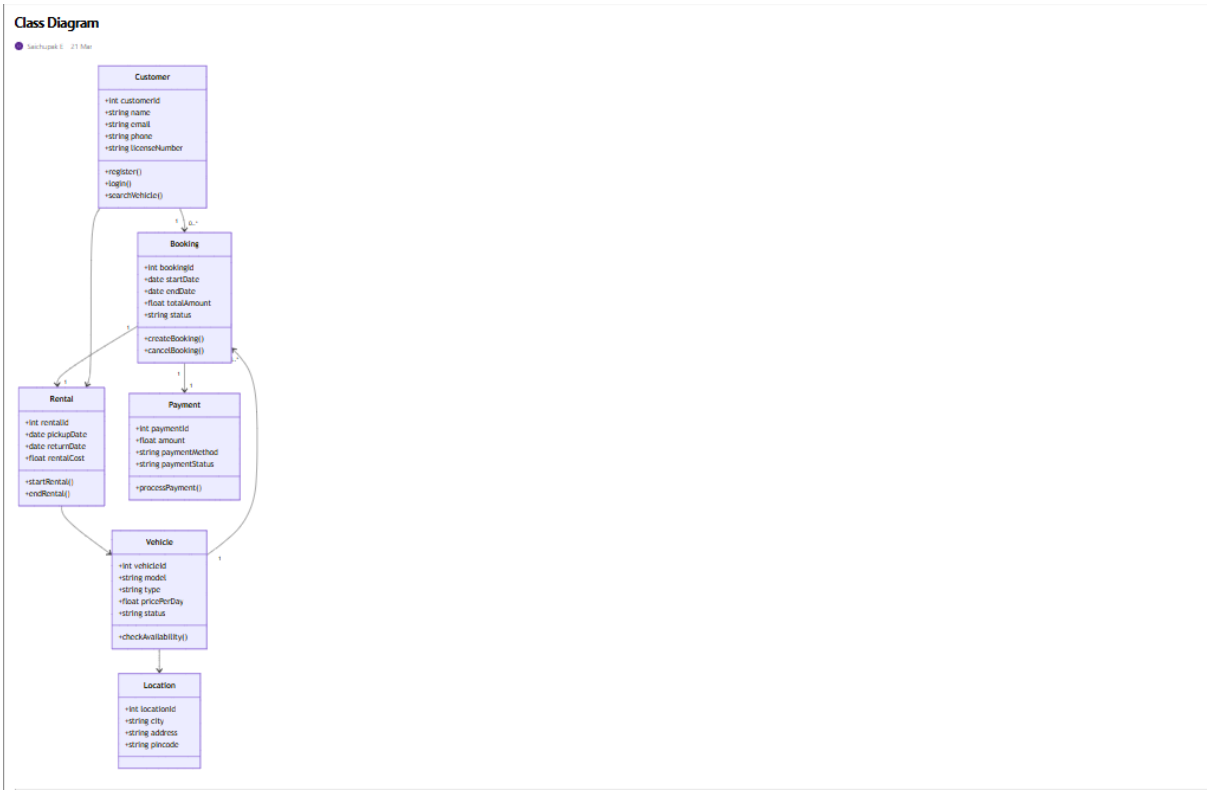
+bookTicket()

}

class Ticket {

+int ticketId

```
  +String seatNumber
  +float price
  +generateTicket()
}

class Payment {
  +int paymentId
  +float amount
  +String status
  +processPayment()
}

%% Relationships
Admin --> Bus : manages
Admin --> Route : manages
Admin --> Schedule : manages

Bus --> Driver : assigned to
Bus --> Schedule : follows

Route --> Schedule : defines

Passenger --> Ticket : books
Ticket --> Payment : includes
Ticket --> Schedule : for
:::
```



Result:

Therefore the class Diagram is executed successfully

Ex no:7 ER diagram

Aim:

To create the ER Diagram for the Online vehicle rental Management system

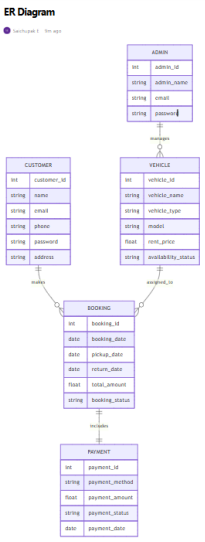
Procedure:

::: mermaid

erDiagram

```
  CUSTOMER {
    int customer_id
    string name
    string email
    string phone
    string password
    string address
  }

  VEHICLE {
    int vehicle_id
    string vehicle_name
    string vehicle_type
    string model
    float rent_price
    string availability_status
  }

  BOOKING {
    int booking_id
```

```
    date booking_date
    date pickup_date
    date return_date
    float total_amount
    string booking_status
  }

  PAYMENT {
    int payment_id
    string payment_method
    float payment_amount
    string payment_status
    date payment_date
  }

  ADMIN {
    int admin_id
    string admin_name
    string email
    string password
  }

  CUSTOMER ||--o{ BOOKING : makes
  VEHICLE ||--o{ BOOKING : assigned_to
  BOOKING ||--|| PAYMENT : includes
  ADMIN ||--o{ VEHICLE : manages
:::
```



Result:

Therefore the above er diagram was executed successfully

Ex No:8 Architecture Diagram

Aim:

To create the Architecture Diagram for the online vehicle rental management system

Procedure:

::: mermaid

graph TD

%% Users

A[Customer / User]

B[Admin]

%% Frontend

C[Web / Mobile UI]

%% Backend

D[Backend Server / API Layer]

%% Modules

E[Authentication Module]

F[Vehicle Management]

G[Booking Management]

H[Payment Module]

I[Notification System]

%% Database

J[(Database)]

%% External Services

K[Payment Gateway]

L[Email / SMS Service]

%% Flow

A --> C

B --> C

C --> D

D --> E

D --> F

D --> G

D --> H

D --> I

E --> J

F --> J

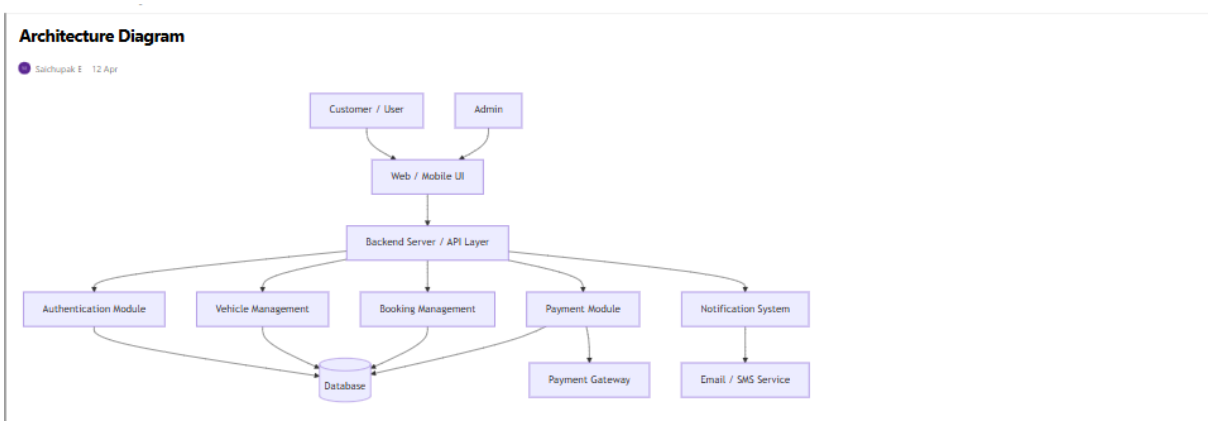
G --> J

H --> J

H --> K

I --> L

:::



Result:

Therefore the Architecture Diagram is executed successfully.

Ex No: 9 Buisness Architecture

Aim:

To create the Buisness Architecture diagram for the online vehicle rental management system

Procedure:

::: mermaid

flowchart TD

A[Customer] --> B[Online Vehicle Rental Management System]

B --> C[User Registration & Login]

B --> D[Vehicle Search]

B --> E[Vehicle Booking]

B --> F[Payment Processing]

B --> G[Booking Confirmation]

H[Admin] --> I[Admin Panel]

I --> J[Manage Vehicles]

I --> K[Manage Customers]

I --> L[Manage Bookings]

I --> M[View Payments]

I --> N[Generate Reports]

E --> O[Vehicle Database]

F --> P[Payment Database]

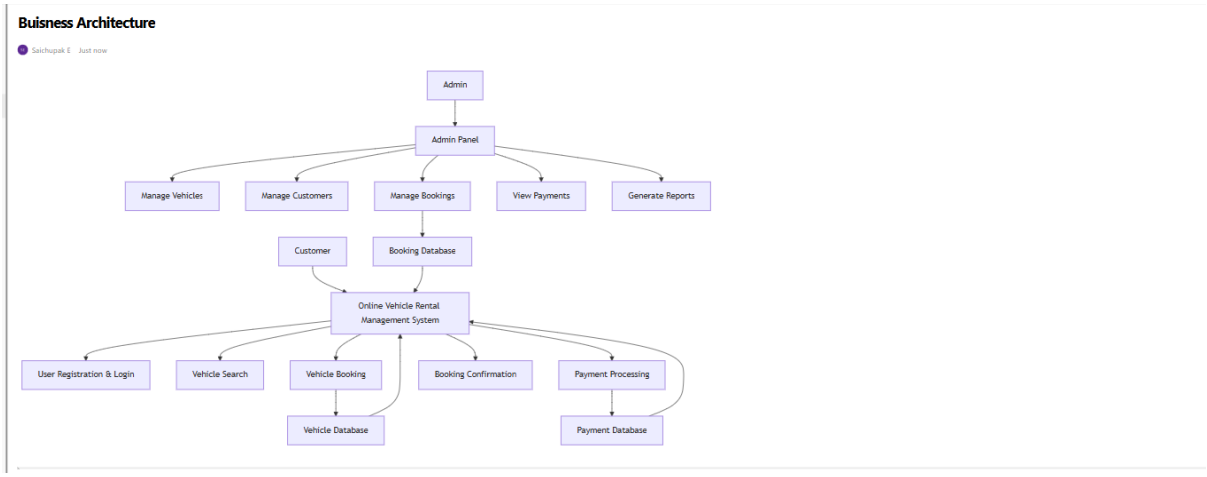
L --> Q[Booking Database]

O --> B

P --> B

Q --> B

:::



Flow Diagram6

Result:

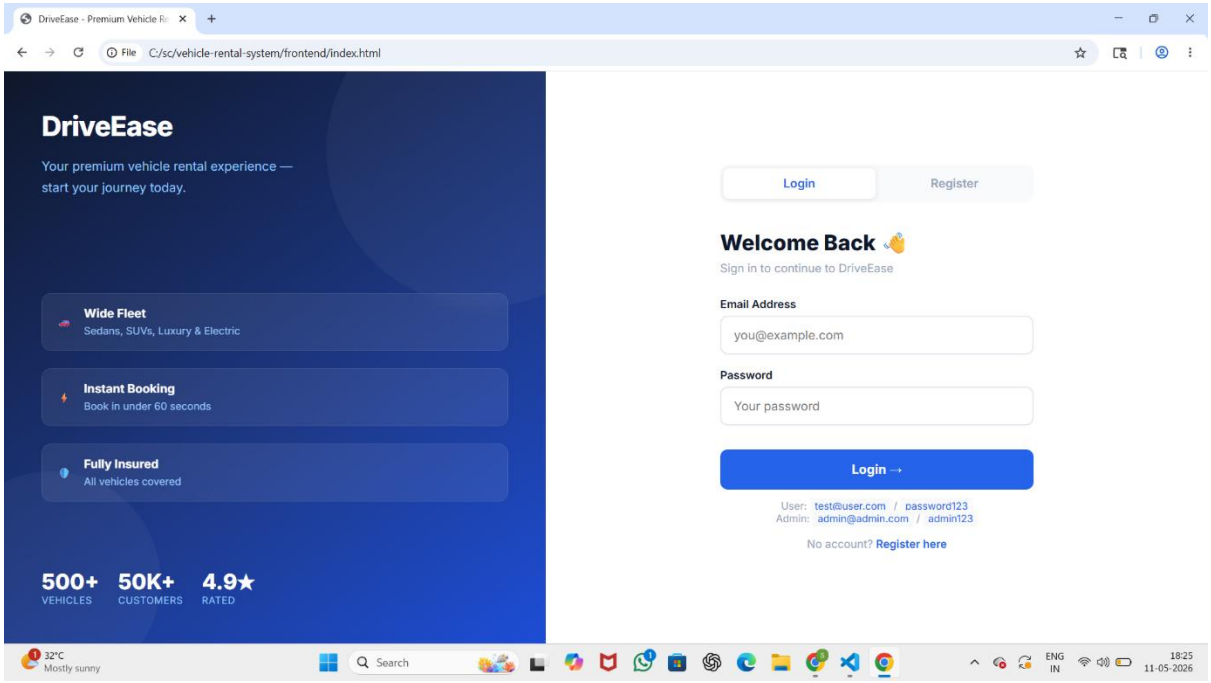
Therefore the Flowchart has been I executed Successfully

Ex No:10 User Interface

Aim:

Design User Interface for the given project

Procedure:



Result:

Threfore the user interface and design is executed Successfully393